Session 26 Teamwork — A Real Sprint Lifecycle

Context

Work in your team (six teams of four students). LIVE already built real Issue CRUD, the generic `IssueEditPanel` pattern, and a real persisted drag-and-drop Board. Your job: build a real Sprint feature on top of that — a form to create a sprint, a page listing every sprint with its own real progress, and real Start/Complete controls — using the identical real patterns already demonstrated (the Server-Action-form shape, a client/server page fetch), not inventing new ones.

Goal

A real `/sprints` page per project where a team can create a sprint, see every sprint's own real points summary, and move a sprint forward through its real lifecycle (`planned` → `active` → `completed`), reachable from Project Detail.

What to build

- A real new Server Action that validates a sprint's name is present (goal is optional) before ever calling the backend — the same validate-first shape `createIssueAction` already uses.
- Two more real Server Actions (start, complete) that call the backend directly and throw on failure — these are NOT submitted from a `<form>`, so think about why they can't follow the identical shape as the create action.
- A real form component, built the same way `CreateIssueForm` was built live: start with the wiring, then add each field one at a time.
- A real page that fetches a project's own sprints and lists them, each showing its own real points summary straight from the backend — never recomputed from a separately-fetched issue list.
- A real component offering exactly ONE lifecycle button at a time, decided by the sprint's own current status — think about what should be offered for "planned," "active," and "completed."
- A real link from Project Detail to the Sprints page.

Rules

Allowed: everything already shown LIVE this session (`"use server"`, `useActionState`, `apiFetch`, the generic Server-Action-form pattern), plus the real backend's own Sprint routes (already built before this session — read their real shape, don't guess it).

Not yet: no priority/assignee/story-points editing on `IssueEditPanel`, no delete-issue flow, no Backlog page — all of that is this session's own homework, not teamwork.

Test it

inside api/

BASH

```
PORT=3000 npm run start
```

inside web/ (second terminal)

BASH

```
npm run dev
```

in a third terminal

BASH

```
npx tsc --noEmit
```

Then, in a real browser: open a project's `/sprints` page, create a real new sprint, start it, confirm its own real Start button disappears and a real Complete button appears; try starting a SECOND sprint while the first is still active and confirm a real, inline error appears instead of a crash.

Done when

- A real new sprint can be created end to end against the real backend.
- A real sprint can be started, then completed, through the real UI.
- Attempting to start a second sprint while one is already active shows a real error, never a thrown exception or a generic crash page.
- `/sprints` is reachable from Project Detail.
- `npx tsc --noEmit` exits clean.

Working time

About 69 minutes total (TW-01..19).